

# NEXUS-R Architecture

Five subsystems. Six modules. One cross-cutting layer. No orchestration theater.

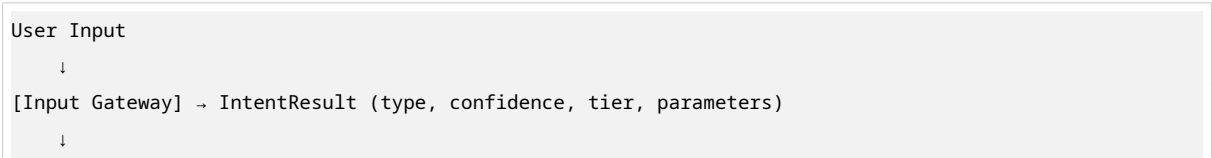
## Design Principles

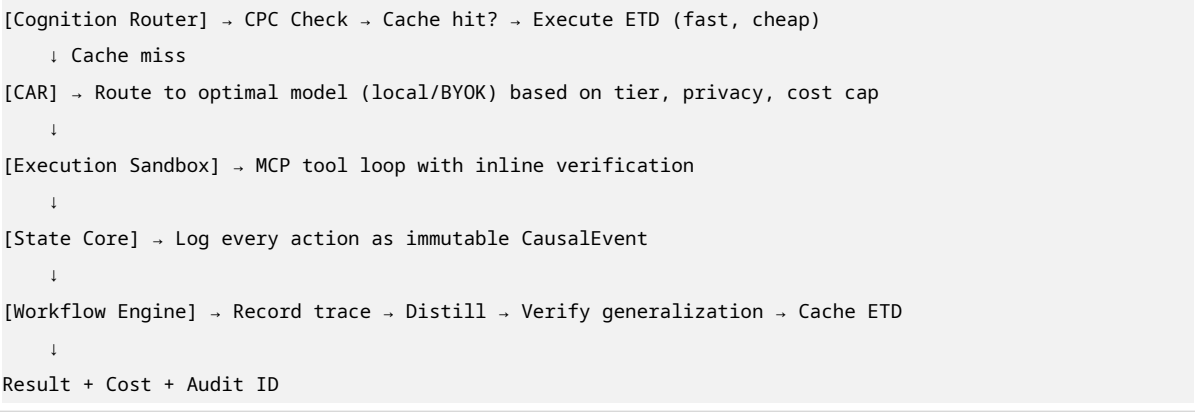
- 1. **Event-sourced everything** — Every state change is an immutable event. The Event Store is the single source of truth.
- 2. **MCP everywhere** — All tool integrations via Model Context Protocol. No custom connectors.
- 3. **LiteLLM for model abstraction** — Unified interface for 100+ providers. We do not build our own provider router.
- 4. **Deny-first security** — ML risk classifier embedded in execution. Default is deny; allow is earned.
- 5. **Privacy as hard constraint** — Local models always preferred when privacy flag set. Not a preference.
- 6. **Explicit generalization bounds** — No workflow applied without >90% proven success rate.
- 7. **CPC short-circuit before routing** — Check cache first. Primary cost-saving mechanism.
- 8. **Inline verification** — Each step has a verify phase. Catches errors at point of occurrence.

## Subsystem Overview

| # | Subsystem                      | Module           | Key Responsibilities                                                                                                               |
|---|--------------------------------|------------------|------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Input Gateway                  | Intent Parser    | Intent classification, confidence estimation, parameter extraction, complexity scoring (T1-T4)                                     |
| 2 | Cognition Router               | ISE + CAR        | Capability profiling, Pareto routing (cost × latency × privacy), CPC short-circuit, adaptive fallback, quality tracking            |
| 3 | Execution Sandbox              | Execution Engine | Sandboxed agent loop via MCP, inline verification, ML risk classification, permission tiers 1-4, event-sourced traces              |
| 4 | State Core                     | Memory Store     | Event log (append-only), 3-tier memory (volatile/durable/identity), causal chaining, vector embeddings, projected views            |
| 5 | Workflow Engine                | CPC/ETD          | Trace recording, distillation, generalization verification (>90%), parameterization, conservative application (>85%), invalidation |
| — | Trust Layer<br>(cross-cutting) | —                | Permission enforcement, sandbox isolation, audit log, cost dashboard, rollback, secret handling                                    |

## Data Flow





## Key Subsystem Details

### 1. Input Gateway

**Responsibility:** Transform natural language into structured intent.

**Components:**

- `IntentParser.parse(user_input: str) → IntentResult`
- `TaskClassifier.classify(intent_result: IntentResult) → TaskTier`
- `ParameterExtractor.extract(user_input: str, intent_type: str) → dict`

**Complexity Scoring (T1–T4):**

| Dimension         | Weight | Measures                                     |
|-------------------|--------|----------------------------------------------|
| Reasoning depth   | 40%    | Logical connectors, ambiguity, planning need |
| Knowledge breadth | 30%    | Domain specificity, recency requirements     |
| Tool complexity   | 30%    | Number/type of tools, irreversibility        |

**Phase 1:** Regex + keyword classification. **Phase 4:** Fine-tuned logistic regression.

### 2. Cognition Router (CAR)

**Responsibility:** Select the optimal model and execution path.

**The Routing Problem:** “Sufficiency” is undecidable a priori — you only know if a model can handle a task after trying. Aggressive routing to weak models may cost more in failed attempts than using a strong model immediately.

**CAR Solution (7 steps):**

1. **CPC Check** — Query ETD cache. If similarity >85% and success rate >90%, execute cached workflow at \$0.

- 2. **Task Classification** — Score reasoning depth, knowledge breadth, tool complexity → composite T1–T4 tier.
- 3. **Capability Profiling** — Each model maintains dynamic scores (code, math, writing, reasoning), latency percentiles, cost, privacy level.
- 4. **Multi-Objective Routing** — Optimize across capability match, latency, cost, privacy (hard constraint), verification confidence.
- 5. **Adaptive Fallback Chain** — 6 tiers: local\_7b → local\_14b → local\_70b → BYOK\_budget → BYOK\_frontier → managed\_premium. Never skip >1 tier. Managed premium always requires approval.
- 6. **Parallel Probe** — When uncertain, send to T1+T2 simultaneously. If T1 succeeds, discard T2.
- 7. **De-escalation Learning** — If T1 consistently handles T2-classified tasks, update classifier to route similar tasks to T1.

Performance targets:

- Routing overhead: <50ms
- Cost reduction vs. always-frontier: 60–80%
- Local mode capability: 80% of cloud on common tasks

3. Execution Sandbox

Responsibility: Safely execute tool invocations with verification.

Sandbox layers:

- **Terminal:** Docker container with workspace mount only. Fallback: subprocess with restricted permissions.
- **Filesystem:** Scoped to workspace directory. Parent directory traversal blocked.
- **Browser (Phase 3):** Isolated Chromium via Playwright. No host filesystem, no real profile.
- **API calls:** Domain whitelisting. Credential injection via SecretRegistry.

Inline verification after every step:

```
verify_step(step: ExecutionStep, result: ToolResult) → VerificationResult
# Checks: exit_code == 0, file_exists, content_matches, status == 200
```

Permission tiers:

| Tier | Actions                                                                     | Approval    |
|------|-----------------------------------------------------------------------------|-------------|
| T1   | Read files, read-only terminal, web search, safe MCP                        | Auto        |
| T2   | Write workspace files, sandboxed terminal, HTTP GET approved                | Auto + log  |
| T3   | Arbitrary terminal, external files, HTTP POST/PUT/DELETE, browser, API keys | User prompt |

Table 3 – continued

| Tier | Actions                                                              | Approval                  |
|------|----------------------------------------------------------------------|---------------------------|
| T4   | Delete files, access secrets, deploy to prod, financial transactions | Explicit confirm + reason |

#### 4. State Core (Three-Tier Memory)

**The v7 insight:** The Event Store should be the single source of truth. All other memory tiers are derived views.

| Tier | Name     | Stores                                                                                          | Lifecycle                    | Storage                |
|------|----------|-------------------------------------------------------------------------------------------------|------------------------------|------------------------|
| 1    | Volatile | Working State — current task context, active plan, tool results                                 | Dies when task completes     | In-memory              |
| 2    | Durable  | Event Store — immutable log of all actions, decisions, outcomes, ETD derivations                | Append-only, 90 days default | SQLite + optional sync |
| 3    | Identity | User Profile — preferences, tool configs, API key refs, behavioral policies, classifier weights | Slow-changing, user-editable | Local encrypted file   |

#### Key features:

- **Causal chaining:** `parent_event_id` links events into provenance chains for precise replay.
- **Vector embeddings:** Semantic retrieval across entire execution history.
- **Projected views:** Read-optimized representations for UI, billing, audit, debugging — no state duplication.
- **Compaction:** Deep Memory-style semantic compaction targeting ~71% token reduction.

#### 5. Workflow Engine (ETD)

**Core innovation:** Execution Trace Distillation with verified generalization bounds.

#### What ETD is NOT:

- NOT “cognitive compression” (vague, unmeasurable)
- NOT “memory” (memory stores information; ETD stores proven execution strategies with quantified reliability)

## ETD Data Structure:

```
{
  "id": "etd_a1b2c3",
  "intent_signature": "deploy-nextjs-to-vercel",
  "intent_embedding": [0.12, -0.05, [0.0, 0.0]],
  "input_schema": {"project_dir": "string", "env_vars": "map"},
  "output_schema": {"deploy_url": "string", "status": "enum"},
  "tool_sequence": [
    {"tool": "terminal", "action": "npm run build", "verify": "exit_code == 0"},
    {"tool": "terminal", "action": "vercel deploy --prod", "verify": "contains(stdout, 'Production')"}
  ],
  "parameter_slots": ["project_dir", "env_vars", "deploy_url"],
  "invariant_checks": ["node >= 18", "vercel-cli installed"],
  "success_count": 14,
  "failure_count": 1,
  "generalization_success_rate": 0.94,
  "avg_cost": 0.0,
  "avg_latency_ms": 4200
}
```

## Seven-Stage Pipeline:

1. **Trace Recording** — Execution Sandbox logs every step as immutable CausalEvent.
2. **Distillation** — Extract minimal causal chain. Drop dead-ends, retries, exploratory steps.
3. **Generalization Verification** — Test on held-out variants. Admit only if >90% success.
4. **Parameterization** — Replace concrete values with typed slots. Merge if >80% step overlap.
5. **Retrieval** — Cosine similarity on embeddings + context gating + ranking by `success_rate` × recency.
6. **Conservative Application** — Opt-in. Environment match required. Abandon on any verification failure.
7. **Invalidation** — Degrade if failure rate >30%, dependency version changes, or 180 days unused.

---

## Cross-Cutting: Trust Layer

**Responsibility:** Enforce security, privacy, and auditability across all subsystems.

### Components:

- **Permission Enforcer** — Deny-first allowlisting. Check before every action.
- **ML Risk Classifier** — Rule-based initially (Phase 2), refined with outcomes (Phase 3+). Target: <10ms per classification.
- **Audit Logger** — Append-only. Fields: timestamp, action\_type, tool, input/output summary, model, tier, cost, approval, hash.
- **Cost Tracker** — Per-task, cumulative, by model, by tier, by date range.
- **Secret Handler** — OS credential manager (keyring). Never log, never send to models, validate BYOK keys on entry.

- **Prompt Injection Defense** — Input framing, output validation (no eval()), action allowlisting, anomaly detection.

## Research Positioning

NEXUS-R ETD novelty claim: *Execution trace distillation with verified generalization bounds for autonomous workflow reuse.*

| Prior Art           | What They Do                                                   | What NEXUS-R Adds                                                                                |
|---------------------|----------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| FlowMind            | Distill traces to structured workflows                         | Explicit generalization verification with quantified bounds; conservative application thresholds |
| Shepherd            | Per-step snapshots, trajectory compression, >5× faster forking | Conservative application + causal chaining for provenance; workflow reuse not just compression   |
| OpenHands Condenser | 2× cost reduction via summarization                            | Reusable execution plans (not just compressed history); parameterized templates                  |
| Deep Memory         | 71% token reduction via bio-inspired compression               | Workflow reuse (not just context compression); structural templates with verification            |

## Repository Navigation

| File                               | Content                                         |
|------------------------------------|-------------------------------------------------|
| specs/00_architecture.md           | This document                                   |
| specs/01_input_gateway.spec.md     | Intent Parser interface, types, error codes     |
| specs/02_cognition_router.spec.md  | CAR interface, routing logic, provider profiles |
| specs/03_execution_sandbox.spec.md | Sandbox config, execution step, verification    |
| specs/04_state_core.spec.md        | Event schema, query API, compaction             |
| specs/05_workflow_engine.spec.md   | ETD pipeline, entry schema, retrieval algorithm |
| specs/06_trust_layer.spec.md       | Permission model, audit schema, risk scoring    |
| specs/99_acceptance_criteria.md    | Phase gates and kill criteria                   |

The architecture is intentionally flat — no hierarchical coordination, no orchestration layers, no message buses. Each subsystem communicates through the Event Store. This eliminates synchronization complexity and enables each subsystem to be developed, tested, and deployed independently.